

Alain Godon <alain.godon@univ-angers.fr>
v 1.0.2

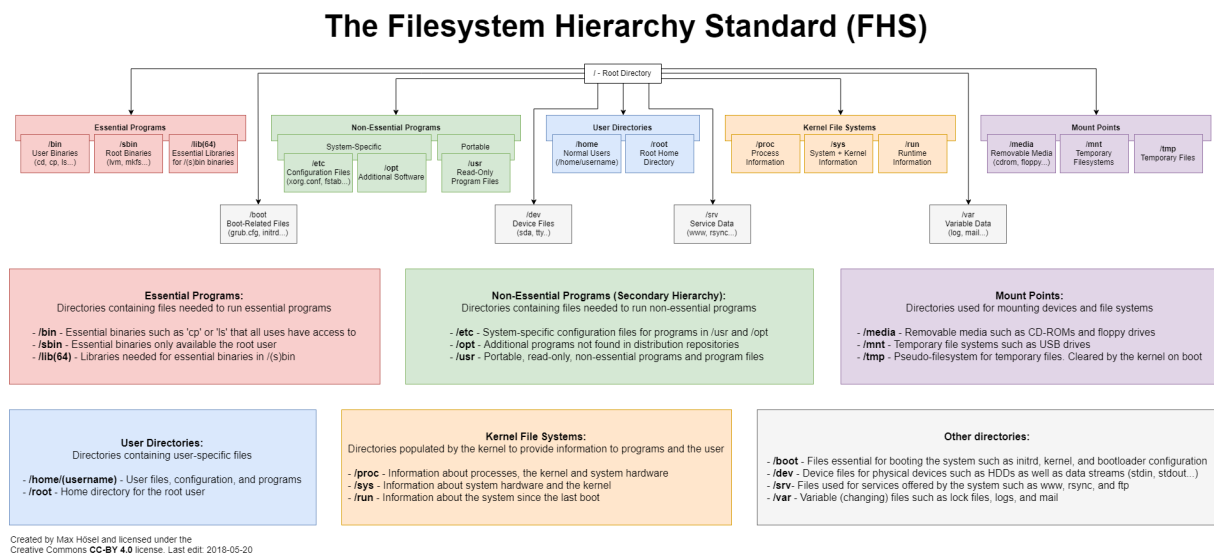
Ce document est sous licence Creative Commons : **CC-BY-NC**.

Ce document est à connaître par coeur si vous envisagez d'écrire *Linux* sur votre CV. Si vous ne maîtrisez que ce document, n'écrivez pas *Linux* sur votre CV.

Ce document est une synthèse à destination des étudiants intéressés par la cybersécurité, vous pouvez utilement consulter **le cours et les exercices** dispensés en 3^{ème} année par **Nicolas Delanoue**.

1 Système de fichiers

La structure est classiquement une arborescence telle que définie par le standard **FHS** où chaque répertoire possède un rôle spécifique :



<https://files.u-angers.fr/data/f-2ec74d5411d71e06.png>

Tout est considéré comme un fichier, que ce soit un "vrai" fichier au sens classique du terme, mais également un répertoire, un périphérique (imprimante, port USB, ...) ou un programme en cours d'exécution.

Ainsi on peut voir (ou modifier selon les droits) des paramètres systèmes :

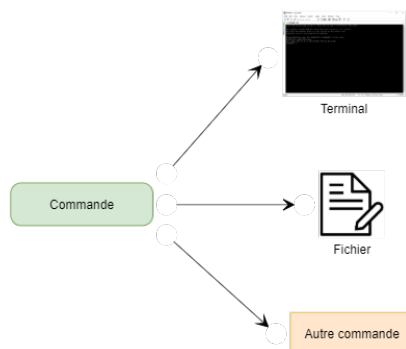
```
agodon@lab:~$ cat /proc/version
Linux version 4.19.0-10-amd64 (debian-kernel@lists.debian.org)
(gcc version 8.3.0 (Debian 8.3.0-6)) #1 SMP Debian 4.19.132-1 (2020-07-24)
agodon@lab:~$ cat /proc/sys/net/ipv4/ip_forward
1
```

Le tableau ci-dessous liste les répertoires les plus importants à connaître :

Répertoire	Contenu	Exemples
bin	programmes essentiels	ls, mkdir
boot	fichiers et programmes pour le démarrage	initrd, vmlinuz
dev	pilotes de périphériques	tty0, null
etc	configuration système	passwd, timezone
home	répertoires des utilisateurs	
lib	bibliothèques de base et modules noyaux	kernel, modules
mnt	point de montage temporaire	
proc	répertoire virtuel pour les infos système (noyaux 2.4 et 2.6)	uptime, version
root	répertoire de l'administrateur	
sbin	programmes système essentiels	mkfs, route
sys	répertoire virtuel pour les infos système (noyau 2.6)	devices, kernel
tmp	répertoire pour fichiers temporaires	
usr	arborescence secondaire	bin, lib
var	données dynamiques	cache, spool

2 Les flux de redirection

Lorsque l'on tape une commande depuis la console, le résultat, sauf cas particulier, s'affiche également dans la console. Il est toutefois possible de rediriger cette sortie vers *autre chose* tel un fichier ou une autre commande, comme le montre le schéma suivant :



2.1 Opérateurs > et >> : redirection vers un fichier

La commande > permet de rediriger la sortie vers un nouveau fichier (un fichier existant sera écrasé), tandis que >> concatène la sortie avec le contenu existant du fichier.

```

agodon@lab:~$ cat f1
Hello
agodon@lab:~$ echo Bonjour > f1
agodon@lab:~$ cat f1
Bonjour
agodon@lab:~$ echo "tout le monde" >> f1
agodon@lab:~$ cat f1
Bonjour
tout le monde
  
```

2.2 Commandes 2>, 2>>, et 2>&1 : redirection des erreurs

Pour compléter la section précédente, on distingue deux types de sortie : la sortie standard (`stdout`) et la sortie d'erreurs (`stderr`). Ces deux flux sont écrits dans la console, mais peuvent être redirigés séparément. Ainsi 2> et 2>> redirigent les erreurs vers un fichier (création ou concaténation).

Supposons qu'en tant qu'utilisateur non privilégié on recherche le fichier `sysctl.conf` sur toute la partition racine avec la commande suivante :

```
agodon@lab:~$ find / -name sysctl.conf
```

On va obtenir en console le résultat attendu mais également énormément de messages d'erreurs pour tous les répertoires dans lesquels on n'a pas les droits de lecture. On peut alors rediriger les erreurs vers `/dev/null` pour ne visualiser que les résultats :

```
agodon@lab:~$ find / -name sysctl.conf 2> /dev/null
```

Enfin, la commande `2>&1` permet de rediriger les deux flux dans une même sortie :

```
agodon@lab:~$ find / -name sysctl.conf > reserror.txt 2>&1
```

2.3 Opérateurs < et << : lire depuis un fichier ou le clavier

La commande `<` permet de rediriger le flux d'entrée (`stdin`) qui est normalement le clavier vers un fichier.

La commande `<<` est très différente puisqu'elle permet la saisie séquentielle depuis le clavier en indiquant un mot-clé (ici `STOP`) qui signifie la fin de saisie :

```
agodon@lab:~/staging$ sort -n << STOP
> 52
> 13
> 6
> 101
> STOP
6
13
52
101
```

On l'utilise plus couramment dans un script de la façon suivante :

```
agodon@lab:~$ cat ex_kbd_redir.sh
#!/bin/bash

cat << EOF
Lignes de texte
Pas besoin de commande "echo"
EOF
agodon@lab:~$ ./ex_kbd_redir.sh
Lignes de texte
Pas besoin de commande "echo"
```

2.4 Opérateur | : enchaînement de commandes

C'est probablement la commande la plus intéressante, appelée `pipe` (on prononce "païpe", on dit également "tube" en bon français), qui permet de rediriger le flux `stdout` d'un programme vers le flux `stdin` d'un autre, etc.

De nombreux exemples de cet opérateur seront utilisés dans la suite.

3 Commandes de base

La syntaxe d'une commande est très souvent de la forme `commande options arguments`. Les options sont souvent précédées d'un ou deux tirets. Par exemple `ls --help` affiche l'aide de la commande `ls`. La liste ci-dessous est à connaître par coeur options comprises : il s'agit du strict minimum.

On appellera sous le terme générique `commande` des instructions qui peuvent être des programmes (par exemple dans les répertoires `/bin` ou `/usr/bin`) comme `/bin/ls` ou des instructions comme `cd` de l'interpréteur de commande

(*shell* comme `/bin/bash`).

En toute rigueur, pour une commande qui est un programme, il faudrait l'appeler avec son nom complet incluant donc le chemin absolu. Lorsque l'on tape une commande qui ne précise pas le chemin absolu, le système interroge la variable `PATH` qui contient une liste de répertoires à consulter pour trouver le programme :

```
agodon@lab:~/tmp$ /bin/ls -l
total 0
agodon@lab:~/tmp$ ls -l
total 0
agodon@lab:~/tmp$ echo $PATH
/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
agodon@lab:~/tmp$ ls -l /bin/ls
-rwxr-xr-x 1 root root 138856 Feb 28 2019 /bin/ls
```

Dans un script, il est toujours prudent de ne pas faire de suppositions sur la variable `PATH` et donc de toujours appeler une commande avec son chemin absolu.

L'opérateur `&` permet, à la fin d'une commande, de la lancer en arrière-plan. Ce sera vu plus en détail dans la suite du document.

3.1 Commandes `cd` et `pwd`

La commande `cd` permet de se déplacer dans l'arborescence tandis que la commande `pwd` permet d'afficher l'emplacement où on se trouve.

Pour accéder à un fichier, on peut donner un chemin absolu depuis la racine comme `/usr/bin/strings` ou un chemin relatif par rapport à l'endroit où on se trouve. On peut alors utiliser les répertoires notés `.` pour le répertoire courant ainsi que `..` pour le répertoire parent. Pour un utilisateur donné, le répertoire `~` désigne son répertoire personnel. La commande `cd` sans argument est l'équivalent de `cd ~` qui renvoie dans le répertoire de base de l'utilisateur.

```
agodon@lab:~$ pwd
/home/agodon
agodon@lab:~$ cd ..
agodon@lab:/home$ pwd
/home
agodon@lab:/home$ cd /home/agodon
agodon@lab:~$ pwd
/home/agodon
```

3.2 Commande `ls`

Cette commande permet de lister tous les fichiers présents dans un répertoire. Les options à connaître sont :

Option	Description
<code>-l</code>	affiche le résultat en format long, notamment les permissions
<code>-a</code>	affiche les fichiers cachés qui commencent par un point
<code>-d</code>	affiche les répertoires sans leur contenu (utile avec <code>*</code>)
<code>-t</code>	affiche les fichiers triés par date de modification
<code>-S</code>	affiche les fichiers triés par taille
<code>-r</code>	inverse l'ordre de tri d'un affichage
<code>-R</code>	affiche récursivement les fichiers

Les options peuvent être combinées :

```
agodon@lab:/etc$ cd /etc
agodon@lab:/etc$ ls -l pa*
-rw-r--r-- 1 root root 552 Feb 14 2019 pam.conf
-rw-r--r-- 1 root root 7 May 25 2020 papersize
-rw-r--r-- 1 root root 1952 Nov 23 17:02 passwd
-rw-r--r-- 1 root root 1947 Nov 23 17:02 passwd-
```

```

pam.d:
total 80
-rw-r--r-- 1 root root 384 Sep 27 2017 chfn
-rw-r--r-- 1 root root 92 Sep 27 2017 chpasswd
-rw-r--r-- 1 root root 581 Sep 27 2017 chsh
-rw-r--r-- 1 root root 1208 Dec 8 11:02 common-account
-rw-r--r-- 1 root root 1249 Dec 8 11:02 common-auth
-rw-r--r-- 1 root root 1440 Dec 8 11:02 common-password
-rw-r--r-- 1 root root 1189 Dec 8 11:02 common-session
-rw-r--r-- 1 root root 1154 Dec 8 11:02 common-session-noninteractive
-rw-r--r-- 1 root root 606 Oct 11 2019 cron
-rw-r--r-- 1 root root 4945 Jul 27 2018 login
-rw-r--r-- 1 root root 92 Sep 27 2017 newusers
-rw-r--r-- 1 root root 520 Feb 14 2019 other
-rw-r--r-- 1 root root 92 Sep 27 2017 passwd
-rw-r--r-- 1 root root 270 Jan 15 2019 polkit-1
-rw-r--r-- 1 root root 143 Jan 10 2019 runuser
-rw-r--r-- 1 root root 138 Jan 10 2019 runuser-l
-rw-r--r-- 1 root root 2257 Jan 10 2019 su
-rw-r--r-- 1 root root 137 Jan 10 2019 su-l
-rw-r--r-- 1 root root 317 Apr 27 2020 systemd-user
agodon@lab:/etc$ ls -ldSr pa*
-rw-r--r-- 1 root root 7 May 25 2020 papersize
-rw-r--r-- 1 root root 552 Feb 14 2019 pam.conf
-rw-r--r-- 1 root root 1947 Nov 23 17:02 passwd-
-rw-r--r-- 1 root root 1952 Nov 23 17:02 passwd
drwxr-xr-x 2 root root 4096 Dec 8 11:02 pam.d

```

3.3 Commande alias

Cette commande est disponible avec l'interpréteur de commandes **bash** et permet d'associer une commande à un nom en général plus court ou plus parlant. On définit un alias selon la syntaxe **alias court='grande commande'**. Une liste d'alias se trouve souvent dans le fichier `~/.bashrc` comme par exemple :

```

alias ll = 'ls -ld'
alias grep='grep --color=auto'

```

La commande **alias** sans argument liste tous les alias actifs. Pour supprimer un alias, on utilise la commande **unalias**.

3.4 Commandes mkdir et rmdir

Ces commandes permettent de créer et de supprimer un répertoire.

```

agodon@lab:~$ ls rien
ls: cannot access 'rien': No such file or directory
agodon@lab:~$ mkdir rien
agodon@lab:~$ ls -ld rien
drwxr-xr-x 2 agodon agodon 4096 Oct 16 09:11 rien
agodon@lab:~$ rmdir rien

```

Un répertoire doit être vide (attention aux fichiers cachés) avant d'être supprimé. Pour supprimer un répertoire non vide, voir la commande **rm** ci-dessous.

3.5 Commande cp

Cette commande permet de copier des fichiers ou des répertoires avec l'option **-r**.

3.6 Commande mv

Cette commande permet de déplacer ou renommer un fichier ou un répertoire.

3.7 Commande rm

La commande `rm` permet de supprimer un fichier ou un répertoire (option `-r`). On utilise aussi l'option `-f` pour supprimer sans demander confirmation.

3.8 Commande ln

La commande `ln` permet de créer des "liens" et on l'utilise principalement pour créer des "raccourcis" avec des liens symboliques (option `-s`) :

```
agodon@lab:~$ echo Hello > unfichier.txt
agodon@lab:~$ ln -s unfichier.txt unraccourci
agodon@lab:~$ ls -l unfichier.txt unraccourci
-rw-r--r-- 1 agodon agodon  6 Oct  2 17:28 unfichier.txt
lrwxrwxrwx 1 agodon agodon 13 Oct  2 17:28 unraccourci -> unfichier.txt
agodon@lab:~$ cat unraccourci
Hello
agodon@lab:~$ echo Bonjour > unraccourci
agodon@lab:~$ cat unfichier.txt
Bonjour
```

3.9 Commande echo

Cette commande permet d'afficher un texte. On notera l'option `-n` qui permet de ne pas effectuer le saut de ligne final :

```
agodon@lab:~$ echo Hello; echo World
Hello
World
agodon@lab:~$ echo -n Hello; echo World
HelloWorld
```

3.10 Commandes cat, more et less

La commande `cat` (abréviation de "conCATenate") permet de concaténer plusieurs fichiers dans un seul, même si on l'utilise souvent pour afficher le contenu d'un fichier :

```
agodon@lab:~$ echo A > f1
agodon@lab:~$ echo B > f2
agodon@lab:~$ cat f1 f2 > ff
agodon@lab:~$ cat ff
A
B
```

La commande `cat` est parfois utilisée dans un cadre un peu particulier lorsqu'il s'agit d'injecter du code en entrée d'un programme (on simule un programme qui afficherait son entrée) :

```
agodon@lab:~$ alias un_prog='cat'
agodon@lab:~$ cat <(python -c "print(('x'*10)+'\x41')") | un_prog
xxxxxxxxxxA
```

On peut avoir besoin, une fois le code injecté, de taper quelques commandes : il suffit d'alors d'utiliser la syntaxe de `cat` avec `/dev/stdin` comme deuxième fichier (à noter que le raccourci `-` fonctionne aussi) :

```

agodon@lab:~$ cat <(python -c "print(('x'*10)+'\x41')") | un_prog
xxxxxxxxxxA
agodon@lab:~$ cat <(python -c "print(('x'*10)+'\x41')") /dev/stdin | un_prog
xxxxxxxxxxA
Hello
Hello
^C
agodon@lab:~$ cat <(python -c "print(('x'*10)+'\x41')") - | un_prog
xxxxxxxxxxA
Bonjour
Bonjour
^C

```

Les commandes `more` et `less` permettent la navigation (simple) dans un fichier comportant de nombreuses lignes.

3.11 Commande grep

Cette commande permet de rechercher un motif dans un texte, soit de façon simple, soit à l'aide d'une expression régulière. Les options à connaître sont :

Option	Description
<code>-i</code>	recherche sans prendre en compte la casse
<code>-v</code>	inverse le résultat de la recherche
<code>-r</code>	effectue une recherche récursive

Dans cet exemple on recherche dans le fichier `/etc/services` tout ce qui concerne le protocole LDAP en UDP :

```

agodon@lab:~$ grep -i LDAP /etc/services | grep -v tcp
ldap          389/udp
ldaps         636/udp

```

3.12 Commande sort

Cette commande permet de trier des valeurs. Les options à connaître sont :

Option	Description
<code>-n</code>	trie des valeurs numériques
<code>-r</code>	trie dans l'ordre décroissant
<code>-u</code>	renvoie un seul résultat en cas de doublons

3.13 Commande touch

Cette commande est principalement utilisée pour créer un fichier vide :

```

agodon@lab:~$ ls -l ex_t
ls: cannot access 'ex_t': No such file or directory
agodon@lab:~$ touch ex_t
agodon@lab:~$ ls -l ex_t
-rw-r--r-- 1 agodon agodon 0 Sep  5 10:41 ex_t

```

3.14 Commandes head et tail

Ces deux commandes fonctionnent de la même façon pour afficher soit le début (`head`) soit la fin (`tail`) d'un fichier. Par défaut ce sont 10 lignes qui sont affichées, ce qui peut être modifié par le paramètre `-n` :

```

agodon@lab:~$ head /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin

```

```
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
agodon@lab:~$ head -n5 /etc/passwd | tail -n2
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

Il faut noter le paramètre `-f` pour la commande `tail` qui permet un affichage dynamique de la fin d'un fichier, très utile pour un fichier de log par exemple :

```
tail -f fichier_de_log
```

3.15 Commandes file et stat

La commande `file` permet de déterminer le type d'un fichier et éventuellement d'autres informations :

```
agodon@lab:~$ file *
5A-cyber:          directory
auto_start_on_boot.sh: Bourne-Again shell script, ASCII text executable
coordinates.png:   PNG image data, 1094 x 1094, 8-bit/color RGBA, non-interlaced
ex_t:             empty
p1.key:           PEM RSA private key
```

La commande `stat` donne d'autres informations plus orientées système :

```
agodon@lab:~$ stat /etc/passwd
  File: /etc/passwd
  Size: 1952          Blocks: 8          IO Block: 4096   regular file
Device: 802h/2050d   Inode: 15597741   Links: 1
Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: (   0/   root)
Access: 2021-01-04 17:03:04.548516588 +0100
Modify: 2020-11-23 17:02:48.499764133 +0100
Change: 2020-11-23 17:02:48.499764133 +0100
 Birth: -
```

3.16 Commande id

Un utilisateur peut connaître son UID grâce à la commande `id` :

```
agodon@lab:~$ id
uid=1000(agodon) gid=1000(agodon) groups=1000(agodon)
```

3.17 Commandes ps et kill

La commande `ps` permet d'afficher les processus en cours, soit dans le `shell` courant sans paramètre, on peut utilement utiliser `-x` pour afficher tous les processus de l'utilisateur :

```
agodon@lab:~$ ps
  PID TTY          TIME CMD
 17673 pts/10    00:00:01 bash
 30938 pts/10    00:00:00 ps
agodon@lab:~$ ps -x
  PID TTY          STAT TIME COMMAND
  5435 ?           Ss    0:00 gpg-agent --homedir /home/agodon/.gnupg --use-standard-socket --daemon
 17673 pts/10    Ss    0:01 /bin/bash
 30939 pts/10    R+    0:00 ps -x
```

La commande `kill` permet de tuer un processus en indiquant son PID :

```
agodon@lab:~$ sleep 60 &
[1] 30964
agodon@lab:~$ ps
  PID TTY          TIME CMD
 17673 pts/10    00:00:01 bash
 30964 pts/10    00:00:00 sleep
 30965 pts/10    00:00:00 ps
agodon@lab:~$ kill 30964
agodon@lab:~$
[1]+  Terminated                  sleep 60
```

3.18 Commandes jobs, bg et fg et raccourci clavier CTRL+z

La commande `jobs` permet d'afficher les processus en cours d'exécution, tandis que `fg` (foreground) et `bg` (background) permettent de passer les processus en avant-plan et en arrière-plan. Lorsqu'un processus est en cours d'exécution en avant-plan, le raccourci `CTRL+z` permet de le stopper :

```
agodon@lab:~$ jobs
agodon@lab:~$ sleep 120 &
[1] 30976
agodon@lab:~$ jobs
[1]+  Running                  sleep 120 &
agodon@lab:~$ sleep 240 &
[2] 30977
agodon@lab:~$ jobs
[1]-  Running                  sleep 120 &
[2]+  Running                  sleep 240 &
agodon@lab:~$ fg %1
sleep 120
^Z
[1]+  Stopped                  sleep 120
agodon@lab:~$ jobs
[1]+  Stopped                  sleep 120
[2]-  Running                  sleep 240 &
agodon@lab:~$ bg %1
[1]+ sleep 120 &
agodon@lab:~$ jobs
[1]-  Running                  sleep 120 &
[2]+  Running                  sleep 240 &
agodon@lab:~$ kill %2
agodon@lab:~$
[2]+  Terminated              sleep 240
agodon@lab:~$ jobs
[1]-  Running                  sleep 120 &
```

3.19 Commande screen

Cet utilitaire qui se présente comme un multiplexeur de terminaux est indispensable. Il permet de créer des sessions, de les "détacher" du terminal, puis d'y revenir. La syntaxe `screen -S nom_session` permet de lancer un nouveau shell dans un terminal virtual. On peut détacher ce terminal par la combinaison de touches `CTRL+a d`. Ensuite la commande `screen -r nom_session` permet de revenir à la session.

3.20 Commande cut

Cette commande très simple est principalement utilisée avec les options `-d` qui spécifie un délimiteur et `-f` un numéro de champ :

```
agodon@lab:~$ head -3 /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
agodon@lab:~$ head -3 /etc/passwd | cut -d":" -f6
/root
/usr/sbin
/bin
```

3.21 Commande tar

Cette commande permet de gérer différents formats d'archive : classique, compressée avec gzip ou bzip2.

Syntaxe	Description
<code>tar -cf fichier.tar dossier/</code>	crée l'archive du dossier sans compression
<code>tar -czf fichier.tar.gz dossier/</code>	crée l'archive du dossier compressé avec gzip
<code>tar -cjf fichier.tar.bz2 dossier/</code>	crée l'archive du dossier compressé avec bzip2
<code>tar -xf fichier.tar</code>	extraît l'archive
<code>tar -xzf fichier.tar.gz</code>	extraît l'archive
<code>tar -xjf fichier.tar.bz2</code>	extraît l'archive

3.22 Commande find

Cette commande permet de rechercher un fichier sur le disque, avec de multiples critères comme par exemple un nom :

```
agodon@lab:~$ find /etc -name sysctl.* 2> /dev/null
/etc/sysctl.conf
/etc/sysctl.d
```

On peut aussi spécifier un type (ici `f` pour file) :

```
agodon@lab:~$ find /etc -type f -name sysctl.* 2> /dev/null
/etc/sysctl.conf
```

Il faut également connaître les options `uid` et `perm` qui permettent de spécifier l'utilisateur et les droits des fichiers. Ici on recherche dans `/bin` les fichiers SUID root :

```
agodon@lab:~$ find /bin -uid 0 -perm /4000 2> /dev/null
/bin/su
/bin/umount
/bin/mount
agodon@lab:~$ ls -l /bin/su /bin/umount /bin/mount
-rwsr-xr-x 1 root root 51280 Jan 10 2019 /bin/mount
-rwsr-xr-x 1 root root 63568 Jan 10 2019 /bin/su
-rwsr-xr-x 1 root root 34888 Jan 10 2019 /bin/umount
```

3.23 Commandes chmod, chown et chgrp

La commande `chmod` permet de modifier les permissions d'un fichier ou d'un répertoire. Chaque fichier a trois types d'utilisateurs qui peuvent y accéder : le propriétaire (**user**), le groupe (**group**), et les autres (**others**). Pour chacun de ces types, on peut définir trois autorisations : lecture (**R**ead), écriture (**W**rite), et exécution (**eX**ecute) :

```
agodon@lab:~$ ls -l
total 3908
drwxr-xr-x 12 agodon agodon 4096 Jan 1 15:53 5A-cyber
-rwxr-xr-x 1 agodon agodon 132 Aug 26 18:45 auto_start_on_boot.sh
-rw-r--r-- 1 agodon agodon 331863 Oct 6 06:30 coordinates.png
```

Les commandes `chown` et `chgrp` permettent de changer le propriétaire et le groupe, elles sont rarement employées par un utilisateur lambda mais plutôt par l'administrateur.

3.24 Commandes `su` et `sudo`

La commande `su` permet de changer d'utilisateur selon la syntaxe `su otheruser`, il faut alors saisir le mot de passe de l'utilisateur `otheruser`. Dans certains cas on peut vouloir uniquement donner des droits différents pour certaines commandes, on utilise alors `sudo cmde`. Il faut préalablement que cela ait été configuré, cela n'est pas traité dans ce document.